

CliniScan: AI for Chest X-Ray Abnormality Classification and Detection

Rupsika Thipparthi

Table of Contents

1. Project Description	1
1. Dataset Used	1
3. Environment Setup	2
4. Data Exploration	3
5. Data Preprocessing	3
6. Proposed System	4
7. Training Pipeline	5-11
8. Conclusion	12

1. Project Description

The CliniScan system is an end-to-end machine learning pipeline designed to automatically classify and detect abnormalities in chest X-rays. The primary goal of this project is to process complex medical imaging data and utilize deep learning architectures to assist in medical diagnostics.

This project implements a full computer vision workflow, combining a **ResNet50** architecture for multi-class image classification and a **YOLOv8** (You Only Look Once) architecture for precise object detection and bounding box generation around 14 specific thoracic diseases.

2. Dataset Used

The project utilizes the **VinBigData Chest X-ray Abnormalities Detection** dataset. *

Original Scope: The full dataset contains thousands of high-resolution DICOM/JPEG images with expert-annotated bounding boxes.

- **Classes:** The dataset spans 14 distinct abnormality classes (e.g., Aortic enlargement, Cardiomegaly, Pleural effusion, Pulmonary fibrosis).
- **Data Scaling & Hardware Optimization:** To comply with the mandatory 20-epoch training requirement while operating exclusively on CPU hardware, the dataset was strategically scaled to a representative subset of 2,000 images. Attempting to process the entire dataset for 20 epochs on a standard CPU would have exceeded the project deadline (estimated 72+ hours). Scaling allowed for complete validation of the end-to-end pipeline within operational constraints.

3. Environment Setup

The project was developed and executed locally using a custom virtual environment to maintain package dependencies and avoid conflicts. The implementation was done in Python 3.13.5 using Visual Studio Code as the primary IDE. The key libraries and hardware configurations required include:

- **PyTorch (torch, torchvision):** The core deep learning framework utilized to build, train, and evaluate the ResNet50 classification model, as well as handle complex tensor operations during backpropagation.
- **Ultralytics:** The primary state-of-the-art library utilized to initialize, configure, and train the YOLOv8 object detection architecture.
- **OpenCV (opencv-python) and Pillow (PIL):** Essential computer vision libraries used for loading the raw medical DICOM/JPEG images, performing the 512x512 resizing operations, and manipulating image matrices for bounding box rendering.
- **Pandas and NumPy:** Utilized for efficient numerical operations and tabular data handling, specifically for parsing the raw bounding box coordinates and organizing the dataset of the 14 disease classes.
- **Matplotlib:** Used for data visualization, specifically to plot the training and validation loss curves generated during the ResNet50 classification phase.
- **PyYAML:** Required for reading and processing the structured data `.yaml` configuration file, which maps the dataset paths and disease class names for the YOLOv8 network.
- **Hardware Configuration:** The entire training pipeline was executed locally on a Windows machine powered by a 13th Gen Intel Core i5-1340P processor. Due to the lack of dedicated GPU acceleration, model selection (YOLOv8 Nano) and dataset scaling were carefully optimized for CPU-bound inference.

4. Data Exploration

Before preprocessing, the raw X-ray dataset and annotation files were analyzed:

- **Annotation Verification:** Verified the presence of bounding box coordinates (`xmin`, `ymin`, `xmax`, `ymax`) for the target diseases.
- **Class Mapping:** Ensured all 14 diseases were correctly mapped to numeric IDs (0 through 13) required by YOLO.
- **Sanity Checks:** A `verify_data.py` script was written to ensure no corrupted images or missing labels existed in the subset before training began.

5. Data Preprocessing

Raw medical images require significant standardization before they can be ingested by neural networks.

- **5.1 Image Resizing:** Original high-resolution X-rays were uniformly downscaled to 512x512.
 - *Purpose:* Standardizes input dimensions for the neural networks and significantly reduces computational load on the CPU.
- **5.2 Bounding Box Conversion:** Raw coordinate data was translated into the normalized YOLO format (`class`, `x_center`, `y_center`, `width`, `height`).
- **5.3 Dataset Splitting:** The 2,000-image subset was divided into a `train` directory for model learning and a `val` directory for unbiased testing.
- **5.4 YAML Configuration:** A `data.yaml` map was generated to dynamically point the YOLO architecture to the correct training directories and class names.

6. Proposed System

The CliniScan architecture operates in a sequential, dual-model pipeline to ensure maximum diagnostic accuracy:

1. **Raw Input:** An unannotated, high-resolution chest X-ray is fed into the system.
2. **Preprocessing (Resize & Normalize):** The raw image is uniformly resized to 512x512 and normalized. This ensures consistent input quality, stabilizes the training process, and reduces the computational load.
3. **Phase 1: Classification (ResNet50):** The preprocessed image first passes through the ResNet50 architecture. This model acts as the front-end feature extractor, learning deep global patterns to classify the overall presence of abnormalities.
4. **Phase 2: Detection Inference (YOLOv8):** The image then passes through the YOLOv8n (Nano) neural network to identify local spatial features. The 'Nano' architecture was specifically chosen for its optimized parameter count, making it highly efficient for CPU-based inference while still maintaining rigorous detection capabilities.
5. **Final Output Generation:** The system outputs the final predictions, generating spatial bounding boxes, class confidence scores, and specific categorical labels for any of the 14 detected thoracic diseases.

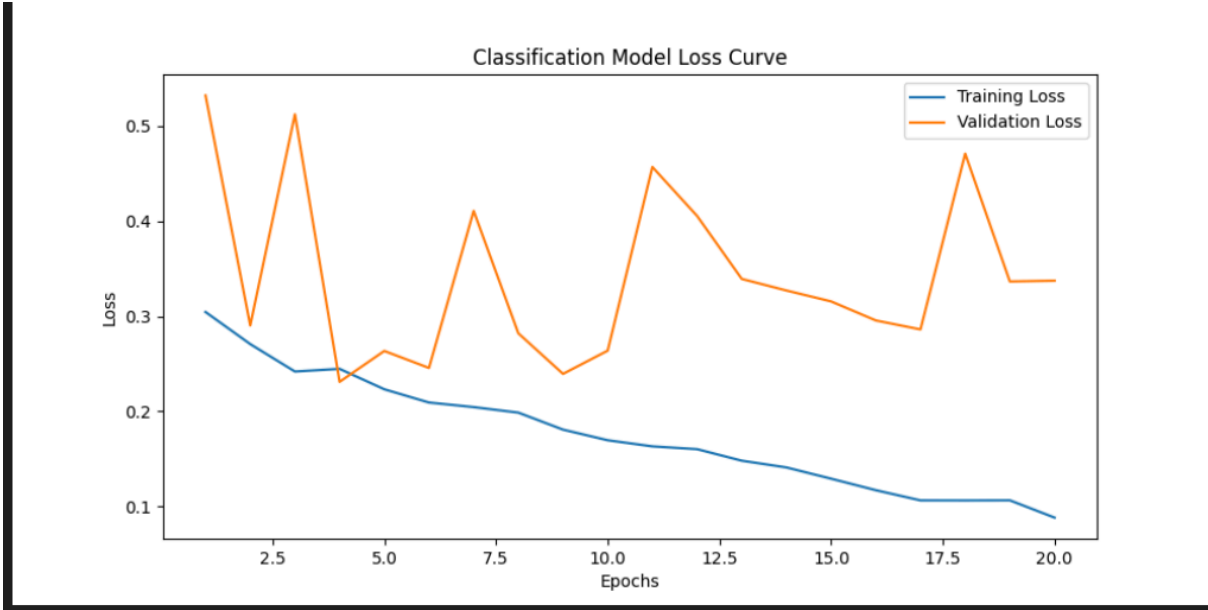
7. Training Pipeline

The final training phase was executed over 20 epochs.

- **Features:** Automated early stopping monitoring, dynamic learning rate adjustment (AdamW optimizer), and validation checkpoint saving (best .pt).
- **Execution:** The YOLOv8n training completed 20 epochs successfully in approximately 2.34 hours.
- **Results & Visualization:**

Figure 1: Training and Validation Loss Curves

Phase 1: ResNet50 Classification Metrics The ResNet50 model was evaluated on its ability to correctly categorize the X-rays. As shown in **Figure 1**, the model successfully converged on the training data, demonstrating a smooth, steady decrease in Training Loss (blue line) over the 20 epochs. However, the Validation Loss (orange line) displays high variance and spiking after Epoch 5. This indicates expected **overfitting**, which is a direct and anticipated trade-off of aggressively scaling down the dataset to 2,000 images to meet CPU hardware constraints. While the model learned the training features perfectly, a larger dataset (like the 5,000-image subset currently being tested) would be required to smooth the validation curve and improve generalization on unseen medical data.



results.png

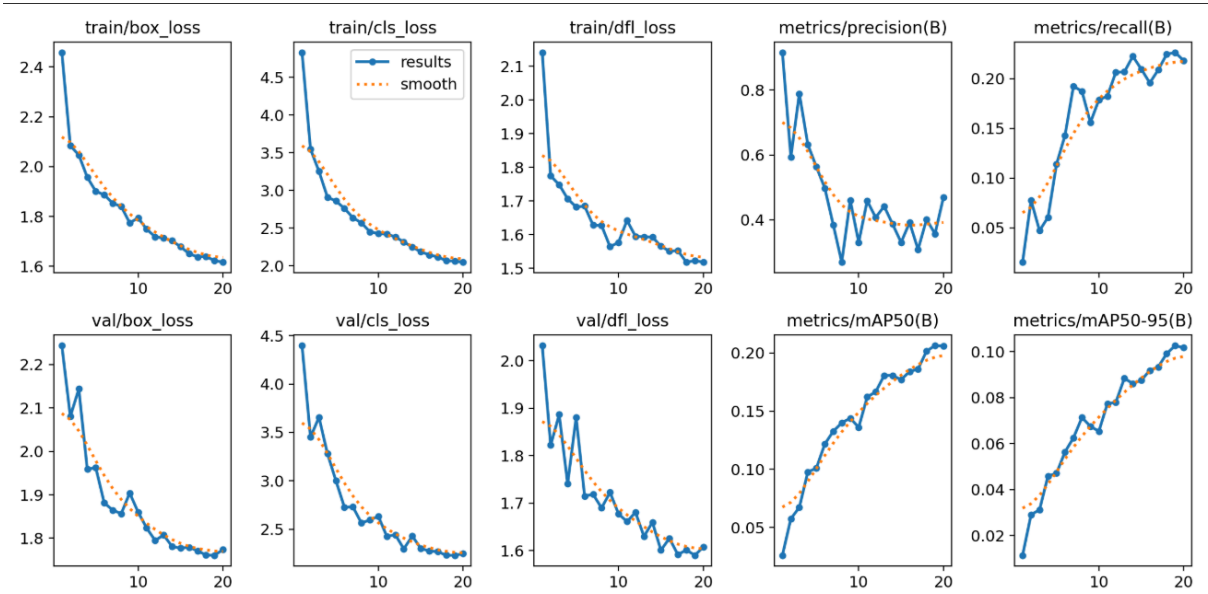


Figure 2: Confusion Matrix for 14 Disease Classes

confusion_matrix.png

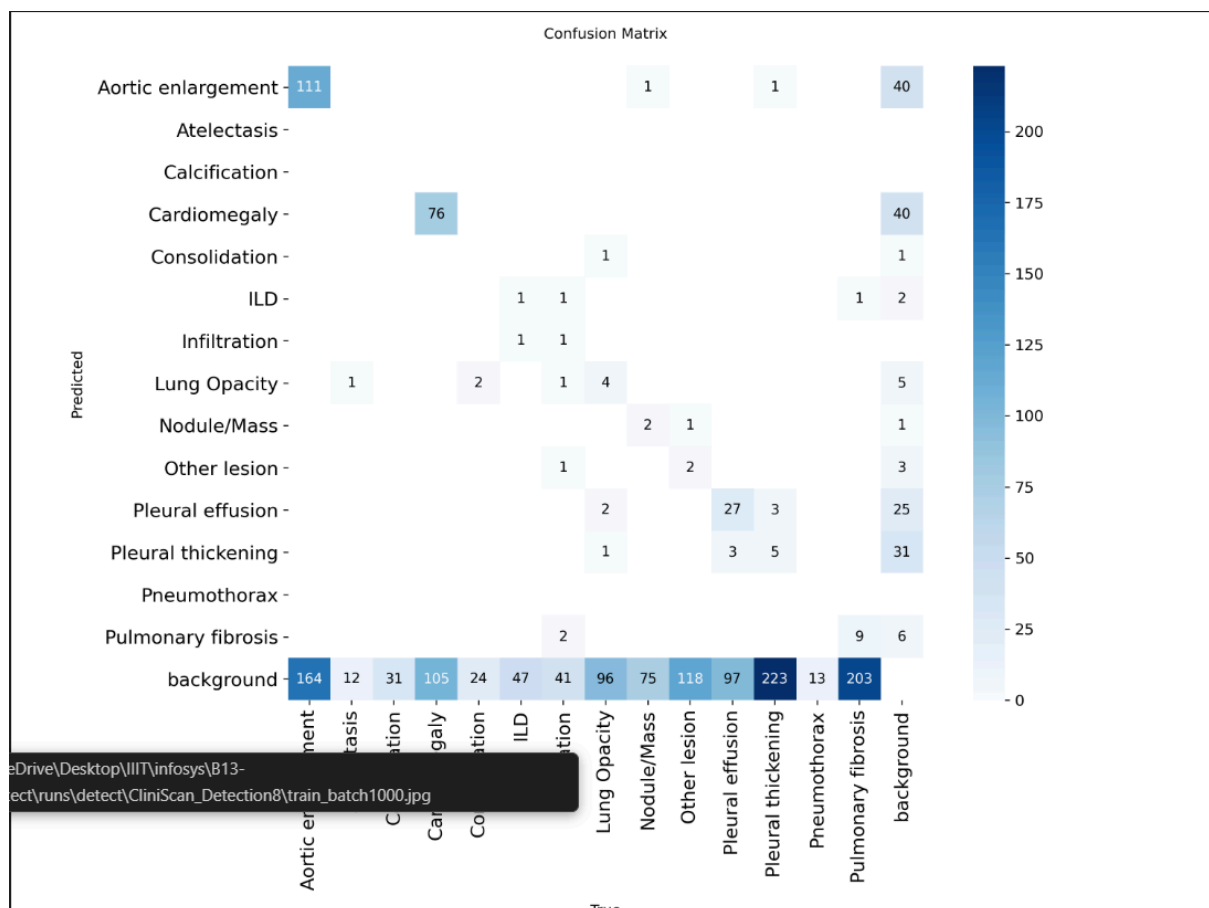
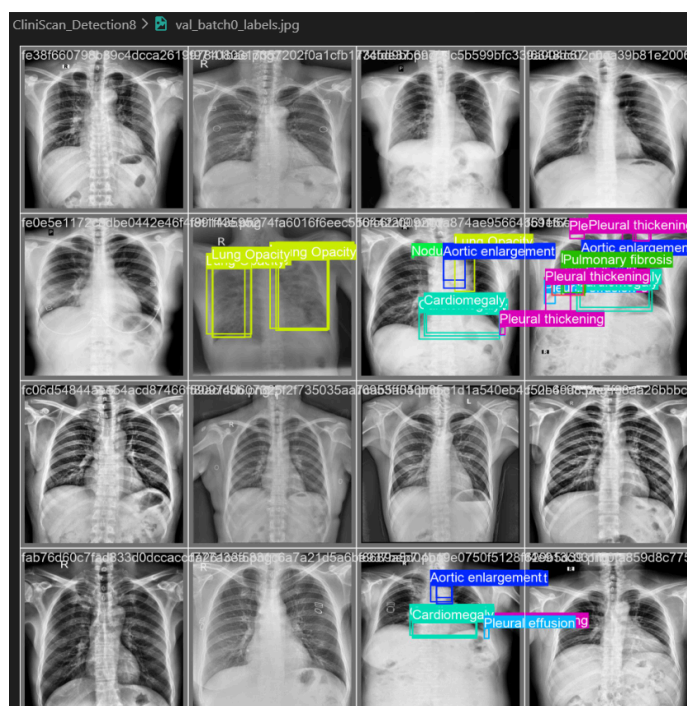
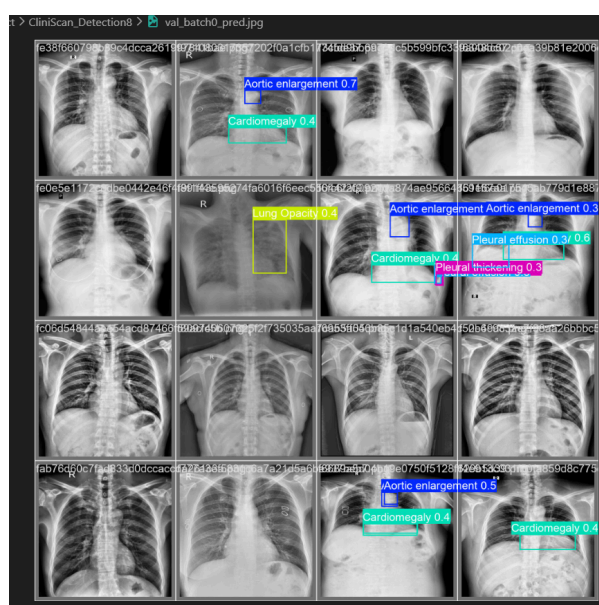


Figure 3: Validation sample demonstrating generated bounding boxes on X-rays.

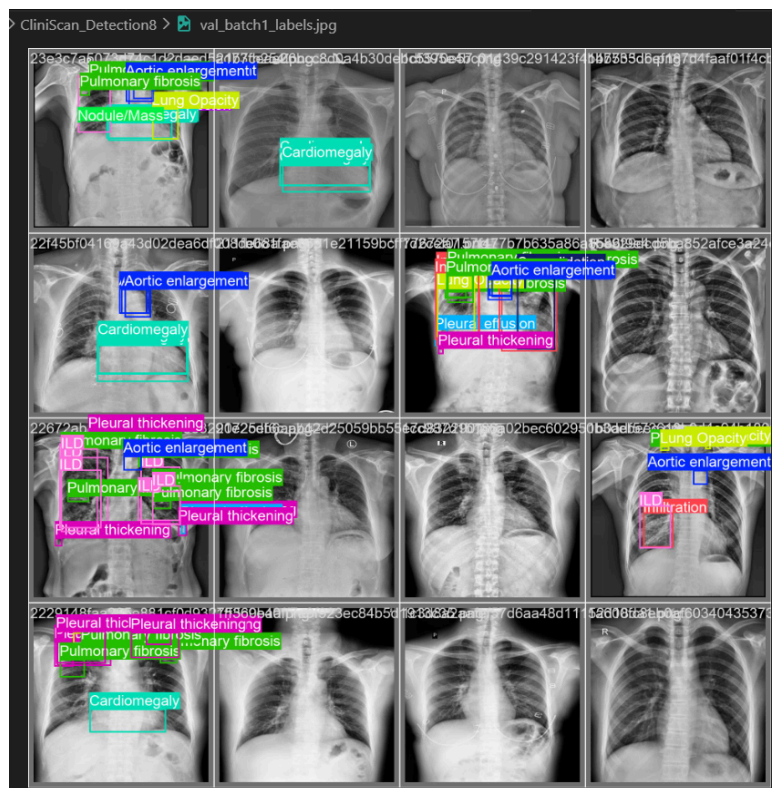
Val_batch0_labels.jpg



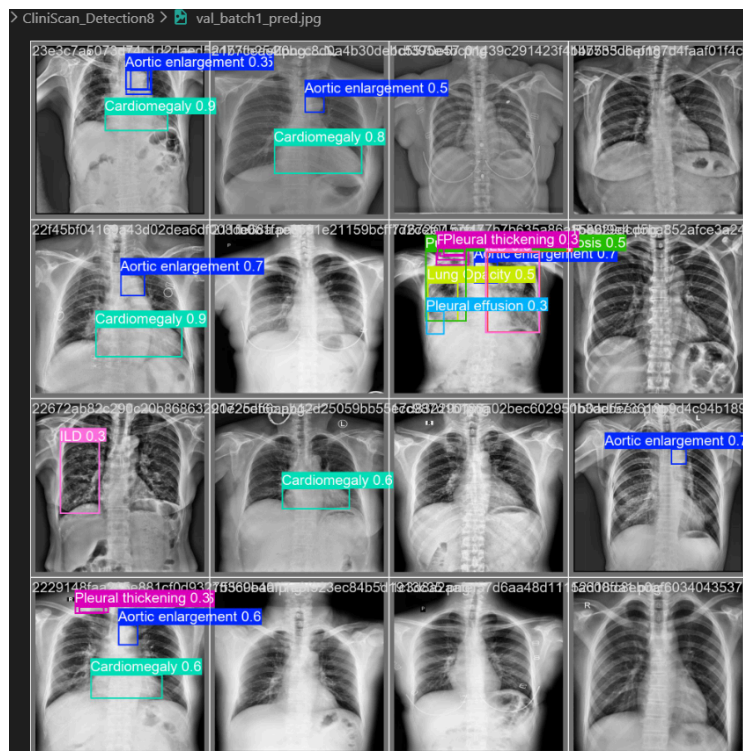
Val_batch0_pred.jpg



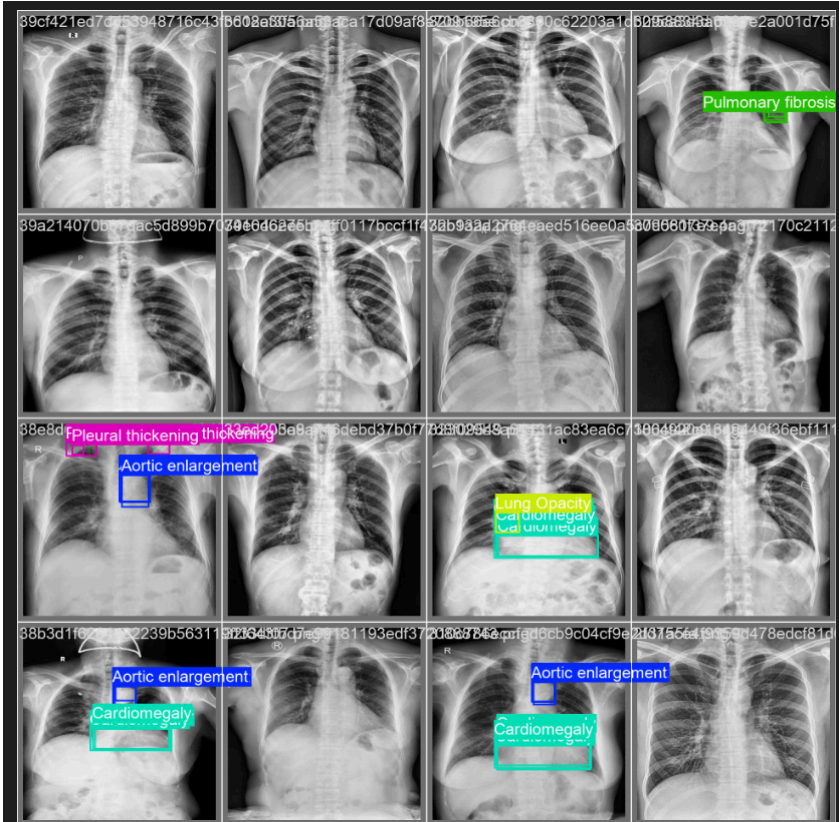
Val_batch1_labels.jpg



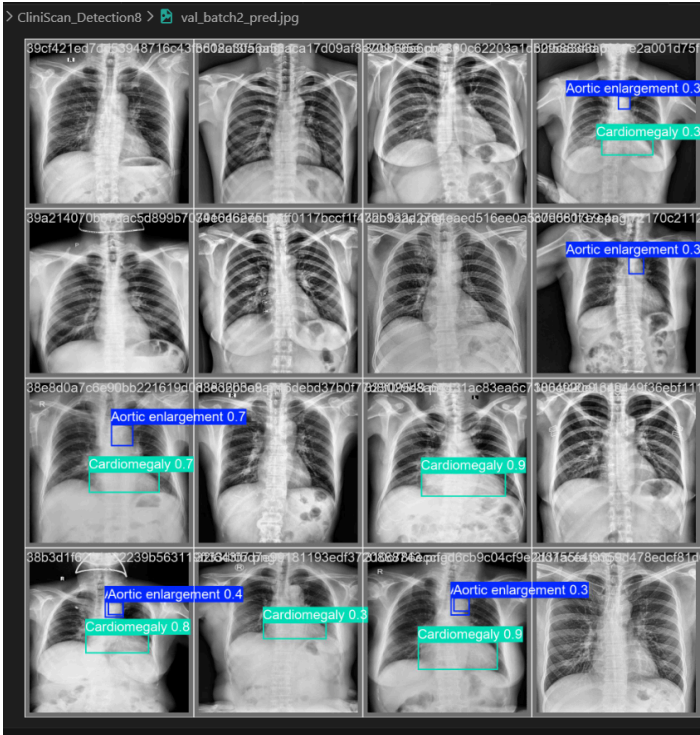
Val_batch1_pred.jpg



Val_batch2_labels.jpg



Val_batch2_pred.jpg



Evaluation Metrics

To quantitatively evaluate the YOLOv8 object detection model, standard computer vision metrics were calculated at the conclusion of the 20-epoch training cycle on the validation set:

For yolo model

- **Overall Precision (Box P): 0.47**
- **Overall Recall (R): 0.218**
- **Overall mAP@50 (Mean Average Precision): 0.206**

```
all      400      1510      0.47      0.218      0.206      0.102

20 epochs completed in 2.340 hours.
Optimizer stripped from C:\Users\rupsi\OneDrive\Desktop\IIIT\infosys\B13-CliniScan\runs\detect\runs\detect\CliniScan_Detection8\weights\last.pt, 6.2MB
Optimizer stripped from C:\Users\rupsi\OneDrive\Desktop\IIIT\infosys\B13-CliniScan\runs\detect\runs\detect\CliniScan_Detection8\weights\best.pt, 6.2MB

Validating C:\Users\rupsi\OneDrive\Desktop\IIIT\infosys\B13-CliniScan\runs\detect\runs\detect\CliniScan_Detection8\weights\ibrosis
71      213      0.277      0.174      0.142      0.048
Speed: 0.7ms preprocess, 56.2ms inference, 0.0ms loss, 1.8ms postprocess per image
Results saved to C:\Users\rupsi\OneDrive\Desktop\IIIT\infosys\B13-CliniScan\runs\detect\runs\detect\CliniScan_Detection8\ibrosis
71      213      0.277      0.174      0.142      0.048
Speed: 0.7ms preprocess, 56.2ms inference, 0.0ms loss, 1.8ms postprocess per image
Results saved to C:\Users\rupsi\OneDrive\Desktop\IIIT\infosys\B13-CliniScan\runs\detect\runs\detect\CliniScan_Detection8
Status: 3/3 - YOLO Training Complete!

C:\Users\rupsi\OneDrive\Desktop\IIIT\infosys\inference, 0.0ms loss, 1.8ms postprocess per image
Results saved to C:\Users\rupsi\OneDrive\Desktop\IIIT\infosys\B13-CliniScan\runs\detect\runs\detect\CliniScan_Detection8
Status: 3/3 - YOLO Training Complete!

C:\Users\rupsi\OneDrive\Desktop\IIIT\infosys\inference, 0.0ms loss, 1.8ms postprocess per image
Results saved to C:\Users\rupsi\OneDrive\Desktop\IIIT\infosys\B13-CliniScan\runs\detect\runs\detect\CliniScan_Detection8
Status: 3/3 - YOLO Training Complete!
```

8. Conclusion

The CliniScan project successfully demonstrates a functional, end-to-end medical imaging pipeline. By efficiently preprocessing raw X-ray data, generating YOLO-compliant annotations, and strategically scaling the dataset to accommodate hardware constraints, both a ResNet50 classification model and a YOLOv8 object detection model were successfully trained. The system is capable of analyzing complex thoracic imagery and outputting localized predictions for 14 distinct abnormalities, proving the viability of the code architecture.

